```c
enum {CSTATE_LISTEN , CSTATE_SYNSENT , CSTATE_SYN_RCVD , CSTATE_ESTABLISHED ,
    CSTATE_FIN_WAIT_1 , CSTATE_FIN_WAIT_2 , CSTATE_TIMED_WAIT ,
    CSTATE_CLOSED , CSTATE_CLOSE_WAIT , CSTATE_LAST_ACK, CSTATE_CLOSING};
typedef struct
{
    bool_t done;    /* TRUE once connection is closed */

    int connection_state;   /* state of the connection (established, etc.) */
    tcp_seq initial_sequence_num;       // this is uint_32 according to #define

    /* any other connection-wide global variables go here */
    uint32_t winsize;
    tcp_seq send_seq_num;       // my data seq, seq_num of the packet that "I" am about to send.
    tcp_seq recv_seq_num;       // peer's data seq, seq_num the "peer" is "expected" to send (us
    tcp_seq to_ack_num;       // how "peer" is doing with "my" packet (use when checking availab

    bool_t acking;
    struct timespec timeout_time;
} context_t;
```

These are the required states, and Struct context_t, with added arguments for window size, my/peer's sequence number, ack number, for delayed ack, and actual timeout time.

```c
void context_t_init(context_t *ctx) {
    ctx->done = FALSE;
    ctx->connection_state = CSTATE_LISTEN;

    ctx->winsize = wsize;
    ctx->send_seq_num = ctx->initial_sequence_num;
    ctx->recv_seq_num = 0;
    ctx->to_ack_num = 1;

    ctx->acking = FALSE;
    return;
}
```

```c
bool_t connected = three_way_handshake(sd, is_active, ctx);

if (connected) {
    ctx->connection_state = CSTATE_ESTABLISHED;
    stcp_unblock_application(sd);

    control_loop(sd, ctx);
}

// still called even if error occur
else {
    stcp_unblock_application(sd);
}
```

Additional function that initializes the context_t struct. And as written in the annotation, to check connection.

```c
bool_t three_way_handshake(mysocket_t sd, bool_t is_active, context_t *ctx) {
    // active open = used by client
    if (is_active) return active_open(sd, ctx);

    // passive open = used by server
    else return passive_open(sd, ctx);
}
```

```c
bool_t active_open(mysocket_t sd, context_t *ctx) {
    // starting as listen

    // 1st handshaking sent
    STCPHeader pheader;
    send_wrapper(sd, ctx, TH_SYN, NULL, 0);
    ctx->connection_state = CSTATE_SYNSENT;     // change state after send.

    // from listen -> synsent

    ctx->send_seq_num++;
    unsigned int event = stcp_wait_for_event(sd, NETWORK_DATA, NULL);

    // just in case
    if (event != NETWORK_DATA)  return FALSE;

    // 2nd handshaking received
    ssize_t got = recv_wrapper(sd, &pheader, NULL, 0);

    if ((pheader.th_flags == (TH_SYN | TH_ACK)) && ctx->send_seq_num == pheader.th_ack) {
        // nothing for send_seq_num since only send ack_num
        ctx->recv_seq_num = pheader.th_seq + 1; // next expected
        ctx->to_ack_num = pheader.th_ack;       // record how much peer acked

        // 3rd handshaking sent
        send_wrapper(sd, ctx, TH_ACK, NULL, 0);
        return TRUE;
    }

    else return FALSE;
}
```

```c
// 1st handshaking received
ssize_t got = recv_wrapper(sd, &pheader, NULL, 0);

// just in case -> might not need here since just in case already in the function
// if (got < 0)  return FALSE;

// 2nd handshaking
if (pheader.th_flags == TH_SYN) {
    ctx->recv_seq_num = pheader.th_seq + 1;
    // no need for send_seq_num or to_ack_num since not sent server's yet.

    ctx->connection_state = CSTATE_SYN_RCVD;

    // 2nd handshaking sent
    send_wrapper(sd, ctx, TH_SYN | TH_ACK, NULL, 0);
    ctx->send_seq_num++;
}

else return FALSE;

event = stcp_wait_for_event(sd, NETWORK_DATA, NULL);    // waiting for connection

// just in case
if (event != NETWORK_DATA)  return FALSE;

// 3rd handshaking received
got = recv_wrapper(sd, &pheader, NULL, 0);
```

3-way handshaking function with active open and passive open.

```c
void send_wrapper(mysocket_t sd, context_t *ctx, uint8_t th_flags, void *segment, size_t seglen) {
    STCPHeader pheader;     // packet header

    pheader.th_flags = th_flags;
    pheader.th_seq = htonl(ctx->send_seq_num);
    pheader.th_ack = htonl(ctx->recv_seq_num);
    pheader.th_win = htons(ctx->winsize);

    // send only header
    if (segment == NULL)    stcp_network_send(sd, &pheader, sizeof(pheader), NULL);

    // send header + data segment
    else    stcp_network_send(sd, &pheader, sizeof(pheader), segment, seglen, NULL);        // automatical

    return;
}
```

```c
ssize_t recv_wrapper(mysocket_t sd, STCPHeader *pheader, char *buffer, size_t buflen) {
    char buf[sizeof(STCPHeader) + STCP_MSS];
    ssize_t len;

    len = stcp_network_recv(sd, buf, sizeof(STCPHeader) + STCP_MSS);

    // just in case
    // if (len < sizeof(STCPHeader))   fprintf(stderr, "wrong recv detected\n");

    memcpy(pheader, buf, sizeof(STCPHeader));
    pheader->th_seq = ntohl(pheader->th_seq);
    pheader->th_ack = ntohl(pheader->th_ack);
    pheader->th_win = ntohs(pheader->th_win);

    if (buffer != NULL) {
        ssize_t to_buf = (len - sizeof(STCPHeader)) > buflen ? buflen : (len - sizeof(STCPHeader));
        memcpy(buffer, buf + sizeof(STCPHeader), to_buf);
        return to_buf;
    }

    else return 0;
}
```

Wrapper functions for send and receive through network layer.

```c
unsigned int event;
STCPHeader recv_header;
char from_appbuf[STCP_MSS];     // application layer -> network layer
char from_netbuf[STCP_MSS];     // network layer -> application layer

/* see stcp_api.h or stcp_api.c for details of this function */
/* XXX: you will need to change some of these arguments! */     // thinking about ANY_EVENT...? + DATO(100ms)
event = stcp_wait_for_event(sd, ANY_EVENT, (ctx->acking ? &ctx->timeout_time : NULL));          // to use minimum malloc
```

Control_loop, used ternary operator to not use malloc if possible. Also, two buffer to clarify received data and data to send.

```c
if (event == TIMEOUT)
{
    send_wrapper(sd, ctx, TH_ACK, NULL, 0);     // handling seq_nums / ack_num already done
    ctx->acking = FALSE;
    continue;    // acking timeout (only available case)
}
```

It used "==" since "&" means 0, which can't go in the if statement.

```c
if ((event & APP_DATA) && (ctx->connection_state == CSTATE_ESTABLISHED || ctx->connection_state == CSTATE_CLOSE_WAIT))
{
    /* the application has requested that data be sent */
    /* see stcp_app_recv() */

    // flow control (space left in the window)
    tcp_seq avail_win = ctx->winsize - (ctx->send_seq_num - ctx->to_ack_num);
    // actually available data in one packet
    tcp_seq avail_psize = avail_win > STCP_MSS ? STCP_MSS : avail_win;

    if (avail_psize > 0) {  // can send packet
        size_t to_send = stcp_app_recv(sd, from_appbuf, avail_psize);

        // just in case
        // if (to_send < 1)   fprintf(stderr, "received 0 or less\n");

        send_wrapper(sd, ctx, TH_ACK, from_appbuf, to_send);
        ctx->send_seq_num += to_send;
        ctx->acking = FALSE;        // might not appropriate, might need it for no multiple ACK.
    }
}
```

Logic to handle application layer signal. CLOSE_WAIT state can still send data

```c
ssize_t just_got = recv_wrapper(sd, &recv_header, from_netbuf, sizeof (from_netbuf));       // pure seg size

// should be always true
if (recv_header.th_flags & TH_ACK) {
    if (recv_header.th_ack > ctx->to_ack_num && recv_header.th_ack <= ctx->send_seq_num) {
        ctx->to_ack_num = recv_header.th_ack;       // got to know renewed window size of peer
        // recv_seq_num in if (just_got) since need segsize.

        // connection state shift
        if (ctx->connection_state == CSTATE_FIN_WAIT_1 && recv_header.th_ack == ctx->send_seq_num) {
            ctx->connection_state = CSTATE_FIN_WAIT_2;
        }

        else if (ctx->connection_state == CSTATE_LAST_ACK && recv_header.th_ack == ctx->send_seq_num) {
            ctx->connection_state = CSTATE_CLOSED;
            ctx->done = TRUE;
            // continue;
        }

        else if (ctx->connection_state == CSTATE_CLOSING) {
            // skipped TIME_WAIT
            ctx->connection_state = CSTATE_CLOSED;
            ctx->done = TRUE;
            // continue;
        }
    }
```

```c
void set_timeout(context_t *ctx) {
    // reference for clock function -> struct timespec count time as nanoseconds
    clock_gettime(CLOCK_MONOTONIC, &ctx->timeout_time);
    // 1000000 nanosec = 1ms, 1000ms = 1sec -> 100000000 nanosec = 100ms
    long int nano_converted = (long int)DATO * (long int)1000000;
    ctx->timeout_time.tv_nsec += nano_converted;
    if (ctx->timeout_time.tv_nsec >= (long int)1000000000) {
        ctx->timeout_time.tv_sec++;
        ctx->timeout_time.tv_nsec -= (long int)1000000000;
    }
    return;
}
```

State change according to what state it is currently in, and function to set timeout time to be after 100ms.

```c
// since always have TH_ACK
if (just_got > 0) {
    if (recv_header.th_seq == ctx->recv_seq_num) {  // correct segment
        stcp_app_send(sd, from_netbuf, just_got);
        ctx->recv_seq_num += just_got;

        if (ctx->acking) {
            send_wrapper(sd, ctx, TH_ACK, NULL, 0);     // got second segment
            ctx->acking = FALSE;
        }
        else {
            ctx->acking = TRUE;
            set_timeout(ctx);
        }
    }
    else {      // think we might not need it (trusted environment)
        if (ctx->acking) {
            ctx->acking = FALSE;
        }
        send_wrapper(sd, ctx, TH_ACK, NULL, 0);     // tell peer this data segment was not expected.
    }
}
```

```c
// TH_FIN arrive when peer reached APP_CLOSE_REQUESTED, which is when app_queue empty = just_got == 0
if (recv_header.th_flags & TH_FIN) {
    if (recv_header.th_seq == ctx->recv_seq_num) {
        stcp_fin_received(sd);
        ctx->recv_seq_num++;

        if (ctx->acking) {
            ctx->acking = FALSE;
        }
        send_wrapper(sd, ctx, TH_ACK, NULL, 0);

        if (ctx->connection_state == CSTATE_ESTABLISHED) {
            ctx->connection_state = CSTATE_CLOSE_WAIT;
        }

        else if (ctx->connection_state == CSTATE_FIN_WAIT_1) {
            ctx->connection_state = CSTATE_CLOSING;
        }

        else if (ctx->connection_state == CSTATE_FIN_WAIT_2) {
            // skip TIMED_WAIT
            ctx->connection_state = CSTATE_CLOSED;
            ctx->done = TRUE;
        }
    }
```

Since TH_FIN means queue is empty, so can send no data, so divided the action that takes when data is received, so only one out of these two will be done.

```c
if (event & APP_CLOSE_REQUESTED)
{
    if (ctx->acking) {
        send_wrapper(sd, ctx, TH_ACK, NULL, 0);
        ctx->acking = FALSE;
    }

    if (ctx->connection_state == CSTATE_ESTABLISHED) {
        // active close
        ctx->connection_state = CSTATE_FIN_WAIT_1;
        send_wrapper(sd, ctx, TH_FIN | TH_ACK, NULL, 0);
        ctx->send_seq_num++;
    }

    else if (ctx->connection_state == CSTATE_CLOSE_WAIT) {
        ctx->connection_state = CSTATE_LAST_ACK;
        send_wrapper(sd, ctx, TH_FIN | TH_ACK, NULL, 0);
        ctx->send_seq_num++;
    }

}
```

This means that 4-way handshake will be held, or being held.

No timed_wait state has been actually used (simplified to close right away, as written in the pdf).

References:

Timeout : https://exeter.tistory.com/1108

Handshaking & data transfer : https://tools.ietf.org/html/rfc793